

수업 07 · 2026-10-22

# OpenCV + 웹캠 실습

이미지의 픽셀, 좌표, 색을 읽고 밝은 부분을 찾습니다. 이미지 파일로 연습한 뒤 웹캠을 켜고 끄는 코드를 살펴봅니다.

10:00~17:00 · 50분 수업 후 10분 휴식 · 중식 11:50~13:00

오늘 필수 활동 10개 · 교시별 설명과 예제를 보고 직접 해 봅니다.

# 2026-10-22 · 처음 시작

기본 경로는 파일 이미지입니다. 개인 실행은 Python 3 설치 후 터미널에서 `python -m pip install opencv-python==4.12.0.88`을 사용할 수 있는 환경이 필요합니다. 설치 권한이 없으면 강사 시연·제공 출력 비교로 진행합니다. 실제 웹캠 스크립트는 강사가 장치·권한·촬영 범위를 확인한 뒤에만 선택해서 실행합니다.

## 파일과 실행

`examples/course/data`는 읽을 입력, `code`는 완성 시연 코드, `work`는 나의 작성·수정 파일입니다. ZIP을 풀고 폴더 구조를 유지합니다. 터미널은 명령을 실행하는 창이며, 에디터는 파일 내용을 쓰고 저장하는 창입니다. 저장만으로 코드가 실행되지는 않습니다.

1. 용어에서 낯선 말을 확인합니다.
2. 교안의 개념과 예를 읽습니다.
3. 활동지의 A01부터 입력을 열고 work 양식에 기록합니다.
4. 코드 실행은 code/line-guide.md의 명령과 조건을 따릅니다.

수업을 마칠 때 확인할 것: 기본 count2와 임계값60의 count3 차이, 입력 실패와 빈장면0 차이, release의 이유를 설명하고 입력값과 결과를 적습니다.

실제 웹캠과 화면 창의 동작은 강의실에서 별도로 확인해야 합니다. 제공 파일을 사용하는 실습과 실제 장치의 결과를 구별해 기록하세요.

## 수업 전, 이 세 가지부터 확인하세요

앞선 내용이 기억나지 않으면 아래 질문부터 읽어 보세요. 먼저 답을 적고 풀이와 비교해 보세요.

### 픽셀

질문: 이미지 폭 100과 높이 60이면 (80,70)은 안쪽인가요?

풀이:  $x=80$ 은 폭 안쪽이지만  $y=70$ 은 높이 범위를 벗어납니다. 좌표는 보통 0부터 세며 배열 접근의 행·열 순서를 확인합니다.

### 검출

질문: 밝은 영역 두 개를 찾으면 물체 두 개라고 확정하나요?

풀이: 그 설정에서 조건을 만족하는 영역 두 개를 찾은 결과입니다. 반사·그림자·불은 영역 때문에 실제 물체와 다를 수 있습니다.

### 비교

질문: 밝기 기준과 최소 면적을 함께 바꾸면 이유를 알 수 있나요?

풀이: 조건을 하나만 바꾸고 나머지를 고정해야 어떤 변화가 결과에 영향을 주었는지 비교하기 쉽습니다.

## 오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

| 교시 | 수업에서 먼저 확인할 활동 | 추가·복습           |
|----|----------------|-----------------|
| 1  | A01, A02, A03  | 새 입력으로 한 번 더 확인 |
| 2  | A04, A05, A06  | 새 입력으로 한 번 더 확인 |
| 3  | A07, A08, A09  | 새 입력으로 한 번 더 확인 |
| 4  | A10, A11, A12  | 새 입력으로 한 번 더 확인 |
| 5  | A13, A14, A15  | 새 입력으로 한 번 더 확인 |
| 6  | A16, A17, A18  | 새 입력으로 한 번 더 확인 |

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

컴퓨터는 사진을 의미가 붙은 사물 목록으로 바로 읽지 않습니다. 이번 수업은 배열의 위치와 색 값에서 시작해, 어떤 조건의 영역을 남겼는지 설명할 수 있는 작은 영상 처리를 만듭니다. 규칙 기반 영상 처리와 학습한 AI 모델의 분류를 혼동하지 않습니다.

## 수업의 출발점

10/8의 데이터 구조와 10/14~15의 변수·조건·반복을 복습합니다. 배열은 줄과 칸으로 값을 담는 방식이며, OpenCV는 이 미지 읽기와 변환을 제공하는 라이브러리입니다. import는 그 도구를 코드에서 쓰도록 가져오는 문장이고 설치와 실행은 별도 단계입니다. 파일 입력을 기본으로 사용하며 실제 카메라는 운영 조건이 갖춰진 경우의 선택 경로로 둡니다.

## 읽는 방법

먼저 개념을 읽고 예제의 입력값과 예상 결과를 살펴봅니다. 수업은 50분 × 6교시로 총 300분이며 중식은11:50~13:00입니다. 각 교시에 개념 설명·시연·직접 실습·풀이가 있습니다. 복습·도전 문제는 기본 실습을 마친 뒤 선택하며, 시간이 부족하면 수업 뒤에 이어서 풀입니다. 학생 실습에 걸리는 시간은 수업 후 기록합니다.

# 1교시 · 이미지와 프레임의 기초

10:00~10:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

## A01 · 이미지는 작은 값들의 격자

### 1교시 필수 개념 보충 · 좌표를 알아야 검사 위치를 정할 수 있다

#### 실무 상황과 원리

포장 검사 담당자가 사진 오른쪽 위의 인쇄만 확인하려고 합니다. 사진 파일을 보았다는 사실과 컴퓨터가 그 위치를 읽었다는 사실은 다릅니다. 파일은 저장된 자료이고, 배열은 읽은 뒤 메모리에서 값이 놓인 표입니다. 격자의 한 칸인 픽셀에 번호를 붙이면 위치를 정확히 전달할 수 있습니다. 왼쪽 위를 출발점으로 가로 번호  $x$ , 세로 번호  $y$ 가 각각 0부터 시작합니다.

OpenCV 컬러 배열의 shape는 높이, 너비, 채널 수 순서입니다. 높이 3, 너비 4, 채널 3이면 (3,4,3)이며 위치는 12곳, 색 성분은 총 36개입니다. `image[y,x]`에서 첫 번호는 행이므로 `image[1,2]`는  $x=2, y=1$ 입니다. BGR 값 `[0,0,255]`는 빨강 성분만 큰 값입니다. RGB 약속을 쓰는 다른 도구로 가져갈 때는 채널 순서를 확인해야 합니다.

완성 예를 종이에 추적합니다. 입력은 shape (3,4,3), 질문은 오른쪽 끝 아래 칸입니다. 처리에서  $x$ 의 마지막 번호는  $4-1=3$ ,  $y$ 의 마지막 번호는  $3-1=2$ 로 계산합니다. 출력 위치는 `image[2,3]`입니다. 번호 4와 3을 그대로 쓰면 범위를 벗어납니다. 해상도만으로 실제 물체가 몇 mm인지 계산하지 못하며 촬영 거리와 기준 길이가 필요합니다.

#### 새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 높이 5, 너비 8인 새 사진의 왼쪽 아래 픽셀과 전체 위치 수를 직접 작성하세요. 채널이 3일 때 색 성분 수도 별도로 쓰세요.

#### 정답과 오답의 이유

정답은 `image[4,0]`, 위치 40곳, 색 성분 120개입니다. `image[0,4]`는 첫 행의 다섯 번째 위치라서 아래 칸이 아닙니다. 120을 물체 개수로 쓰면 색 성분과 검사 대상을 혼동한 것입니다.

#### 확인 질문과 답

1. shape의 첫 값은? → 높이입니다.
2.  $x=3, y=2$ 의 배열 표기는? → `image[2,3]`입니다.
3. BGR의 마지막 값은? → 빨강 성분입니다.

픽셀은 이미지에서 한 위치의 색이나 밝기를 담는 최소 단위입니다. 모눈종이의 칸과 비슷하지만 실제 물체의 고정 길이를 뜻하지는 않습니다.

값의 배열을 이해하면 색이 바뀌거나 좌표가 어긋난 이유를 설명할 수 있습니다. 이미지 파일의 이름만으로 내용과 크기를 알 수 없습니다.

#### 입력에서 살펴볼 예

4×3 회색조 예 0 0 255 255 0 80 80 255 0 0 0 0

가로 4칸, 세로 3칸입니다. 0은 어둡고 255는 밝습니다. 이 예의 배열은 높이 3, 너비 4입니다.

## 구분해야 할 반례

픽셀 한 칸이 현실의 1mm라는 뜻은 아닙니다. 촬영 거리와 렌즈, 해상도에 따라 대응 길이가 달라집니다.

스스로 설명할 질문: 가로·세로와 실제 길이를 왜 구분해야 하나요?

활동 연결: [입력](#) · [기록 양식](#)

## A01 · 직접 해보기

### A01 · 이미지는 작은 값들의 격자

**목적** 값의 배열을 이해하면 색이 바뀌거나 좌표가 어긋난 이유를 설명할 수 있습니다. 이미지 파일의 이름만으로 내용과 크기를 알 수 없습니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A01\\_input.md](#)

작업: [A01\\_worksheet.md](#)

#### 단계

1. 격자에서 밝은 칸 세 개를 표시합니다.
2. 높이와 너비를 서로 다른 칸에 씁니다.
3. 한 픽셀의 실제 길이를 알기 위해 필요한 정보를 적습니다.

**예상 결과** 격자 읽기에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** 가로·세로와 실제 길이를 왜 구분해야 하나요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

### A02 · 좌표와 BGR 채널

좌표는 위치를 가리키는 번호이고 채널은 한 픽셀의 색 성분입니다. 이 수업의 OpenCV 컬러 입력은 파랑·초록·빨강 순서인 BGR입니다.

배열은 image[y,x]로 읽으므로 화면의 x,y 설명과 순서를 구분해야 합니다. 채널 순서를 바꾸면 빨강과 파랑의 해석이 바뀝니다.

#### 입력에서 살펴볼 예

좌표: x=2, y=1 픽셀: [0, 0, 255] 컬러 shape: (240,320,3)

image[1,2]에서 BGR 값 [0,0,255]는 빨강입니다. shape는 높이 240, 너비 320, 채널 3입니다.

## 구분해야 할 반례

[0,0,255]를 모든 도구에서 파랑으로 읽지는 않습니다. RGB와 BGR은 채널 약속이 다릅니다.

스스로 설명할 질문: image[40,30]은 어느 x,y를 가리키나요?

활동 연결: [입력](#) · [기록 양식](#)

## A02 · 직접 해보기

### A02 · 좌표와 BGR 채널

**목적** 배열은 image[y,x]로 읽으므로 화면의 x,y 설명과 순서를 구분해야 합니다. 채널 순서를 바꾸면 빨강과 파랑의 해석이 바뀝니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A02\\_input.md](#)

작업: [A02\\_worksheet.md](#)

#### 단계

1. x,y와 배열 인덱스를 대응합니다.
2. BGR 세 값에서 빨강 성분을 표시합니다.
3. shape 세 숫자의 뜻을 씁니다.

**예상 결과** 좌표 대응에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** image[40,30]은 어느 x,y를 가리키나요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

함께 보는 예와 더 해보기: 파일과 프레임의 차이

[이 교시의 다른 예와 코드 해설 열기](#)

# 2교시 · 읽기·좌표·변환

11:00~11:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

## A04 · 이미지 파일 읽기와 실패

### 2교시 필수 개념 보충 · 잘라내기와 밝기 변환이 바꾸는 정보

#### 실무 상황과 원리

작업대 사진에는 부품과 함께 창문 반사가 들어 있습니다. 검사자는 작업대 가운데만 보고 싶습니다. 관심 영역은 전체 사진에서 필요한 작은 구역이며 ROI라고 부릅니다. 잘라내기는 바깥 픽셀을 제외하고, 크기 변경은 새 격자로 값을 다시 배치합니다. 둘을 같은 일이라고 생각하면 검사 위치와 면적 기준이 틀어집니다.

`image[20:80,30:130]`은 행 20부터 79, 열 30부터 129까지 가져옵니다. 콜론 뒤의 끝 번호는 포함하지 않습니다. 따라서 높이는  $80-20=60$ , 너비는  $130-30=100$ 입니다. 잘린 사진의 (0,0)은 원본의 (30,20)입니다. 원본 좌표가 필요하다면 잘린 x에 30, y에 20을 더합니다. 가로와 세로를 각각 절반으로 줄이면 같은 물체의 면적은 대략 4분의 1이 됩니다. 정확한 경계 값은 다시 만든 픽셀과 윤곽선에 따라 달라집니다.

회색조는 여러 색 성분을 밝기 하나로 바꿉니다. 세 성분의 단순 평균이 아니라 변환 규칙을 사용합니다. 예제의 순수 빨강 BGR [0,0,255]는 대략 76, 초록 [0,255,0]은 150, 파랑 [255,0,0]은 29입니다. 밝은 부품을 찾기에는 편하지만 같은 밝기의 서로 다른 색은 구분하기 어렵습니다. 완성 흐름은 원본 읽기 → 필요한 영역 자르기 → 회색조 변환 → 높이와 너비 기록입니다. 회색조 결과의 shape는 (60,100)이며 채널 3이 사라집니다.

#### 새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: `image[10:50,20:90]`을 새로 해석하고, 잘린 좌표  $x=4, y=7$ 의 원본 좌표를 계산하세요. 빨간 라벨과 녹색 라벨을 구별할 때 회색조만으로 충분한지도 판단하세요.

#### 정답과 오답의 이유

높이 40, 너비 70, 원본  $x=24, y=17$ 입니다. 색이 업무 판단의 핵심이면 원본 색 정보를 보존해야 합니다. 끝 번호를 포함해  $41 \times 71$ 로 쓰거나 모든 색 차이가 밝기 차이로 유지된다고 쓰면 오답입니다.

#### 확인 질문과 답

1. 끝 번호 50은 포함되나요? → 아닙니다.
2. 잘린 좌표의 출발점은? → 잘라낸 영역의 왼쪽 위입니다.
3. 반으로 축소된 면적 기준은? → 기존 기준을 그대로 확정하지 말고 재검증합니다.

`imread`는 파일을 이미지 배열로 읽는 함수입니다. 성공한 배열과 실패한 `None`을 구분한 뒤 다음 처리를 해야 합니다.

읽지 못한 파일을 회색조로 바꾸려고 하면 원인과 멀리 떨어진 오류가 날 수 있습니다. 입구에서 실패를 확인하면 경로·파일 이름을 먼저 복구할 수 있습니다.

#### 입력에서 살펴볼 예

01\_inspect.py 전체 `data/parts.png` 예상 shape (240,320,3)

코드가 파일을 찾고 배열 크기와 픽셀 값을 출력합니다. 회색조 결과는 work/gray.png에 저장합니다.

## 구분해야 할 반례

저장만 누르면 Python 코드가 실행되는 것은 아닙니다. 터미널에서 제공 명령을 실행하고 결과 파일 시각과 내용을 확인합니다.

스스로 설명할 질문: None 확인을 색 변환보다 먼저 하는 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

## 01\_inspect.py 전체

```
import cv2

from pathlib import Path

root = Path(__file__).resolve().parent.parent
source = root / "data" / "parts.png"

image = cv2.imread(str(source))

if image is None:
    raise ValueError("이미지를 읽지 못했습니다: " + str(source))

print("shape:", image.shape)
print("BGR:", image[40, 30].tolist())

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

saved = cv2.imwrite(str(root / "work" / "gray.png"), gray)

if not saved:
    raise OSError("출력 파일을 저장하지 못했습니다")

print("gray:", gray.shape)
```

실행 조건: Python 3와 OpenCV가 설치된 PC. 카메라나 GUI 창을 열지 않는 파일 입력 예제입니다.

실행 폴더: examples/course

명령: `python -X utf8 code/01_inspect.py`

결과:

```
shape: (240, 320, 3)
BGR: [220, 220, 220]
gray: (240, 320)
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: parts.png를 empty.png로 바꾸면 크기는 같고 지정 픽셀 값은 [0, 0, 0]입니다.

오류 복구: ModuleNotFoundError이면 설치 환경을 확인합니다. 읽기 오류면 압축을 풀고 data 폴더와 파일 철자를 확인합니다.

## A04 · 직접 해보기

### A04 · 이미지 파일 읽기와 실패

**목적** 읽지 못한 파일을 회색조로 바꾸려고 하면 원인과 멀리 떨어진 오류가 날 수 있습니다. 입구에서 실패를 확인하면 경로·파일 이름을 먼저 복구할 수 있습니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A04\\_input.md](#) · [01\\_inspect.py](#) · [parts.png](#)

작업: [A04\\_worksheet.md](#)

#### 단계

1. 실행 폴더와 입력 경로를 찾습니다.
2. 코드 각 줄을 읽고 크기·픽셀 값을 예상합니다.
3. 실행 또는 제공 출력과 비교하고 gray.png를 확인합니다.

**예상 결과** 실행 경로에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** None 확인을 색 변환보다 먼저 하는 이유는 무엇인가요?

**오류 해결** cv2 설치 오류와 입력 경로 오류를 구분합니다. 제공 출력 비교로 계속하고 실제 실행 여부를 기록합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

### A05 · 관심 영역과 크기

관심 영역 ROI는 이미지에서 이번 문제에 필요한 부분입니다. 책에서 필요한 문단을 고르는 것처럼 분석할 범위를 먼저 정합니다.

배경까지 처리하면 관계없는 밝은 물체도 함께 잡힐 수 있습니다. 잘라내기와 크기 변경은 목적과 좌표를 바꾸므로 원본 크기를 기록해야 합니다.

#### 입력에서 살펴볼 예

원본 shape (240,320,3) ROI: image[20:80,10:70] resize 목표: (160,120)

슬라이스 끝 번호는 포함하지 않아 ROI는 60×60입니다. resize의 크기 인자는 너비 160, 높이 120입니다.

## 구분해야 할 반례

자르기는 범위를 버리고 크기 변경은 전체를 새 격자로 만듭니다. 둘을 같은 작업으로 설명하면 좌표와 검출 면적이 어긋납니다.

스스로 설명할 질문: ROI 자르기와 resize는 어떤 정보를 바꾸나요?

활동 연결: [입력](#) · [기록 양식](#)

## A05 · 직접 해보기

### A05 · 관심 영역과 크기

**목적** 배경까지 처리하면 관계없는 밝은 물체도 함께 잡힐 수 있습니다. 잘라내기와 크기 변경은 목적과 좌표를 바꾸므로 원본 크기를 기록해야 합니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A05\\_input.md](#)

작업: [A05\\_worksheet.md](#)

#### 단계

1. ROI의 시작·끝 번호를 표시합니다.
2. 결과 높이·너비를 계산합니다.
3. 전체 크기를 반으로 바꾼 후 면적 기준을 그대로 써도 될지 토의합니다.

**예상 결과** ROI 계산에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** ROI 자르기와 resize는 어떤 정보를 바꾸나요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

함께 보는 예와 더 해보기: 회색조로 바꾸면 남는 정보와 사라지는 정보

[이 교사의 다른 예와 코드 해설 열기](#)

## 3교시 · 밝기와 연결 영역

13:00~13:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

### A07 · 임계값과 이진 마스크

**3교시 필수 개념 보충 · 임계값과 면적 기준은 서로 다른 질문이다**

#### 실무 상황과 원리

검사 화면에서 밝은 부품 세 개와 작은 먼지 하나가 보입니다. 먼저 밝은 위치를 고르고, 그다음 너무 작은 영역을 제외하려 합니다. 첫 단계는 어떤 픽셀을 남길지 묻고, 두 번째는 남은 덩어리 중 어떤 크기를 셀지 묻습니다. 이 두 질문에 각각 threshold와 min\_area라는 설정 이름을 붙입니다.

THRESH\_BINARY는 밝기가 임계값보다 클 때 255, 작거나 같을 때 0을 출력합니다. 입력 밝기 [80,100,101,220]과 임계값 100이면 출력은 [0,0,255,255]입니다. 100이 흰색이 되지 않는 이유는 비교가 크거나 같음이 아니라 엄격한 초과이기 때문입니다. 흰색은 조건 통과 위치를 뜻하며 부품 종류나 불량량의 의미까지 알려 주지 않습니다.

윤곽선은 남은 영역의 경계입니다. contourArea는 경계 좌표로 계산한 기하학적 면적이며 흰 픽셀 개수를 그대로 센 값과 같다고 보장하지 않습니다. 면적 [25,100,180]에서 최소 면적 100과 비교 조건  $\geq$ 를 쓰면 두 영역이 남습니다. 첫 밝기 기준의  $>$ 와 면적 기준의  $\geq$ 가 다르다는 점을 코드에서 찾습니다. 붙어 있는 두 부품은 한 윤곽선으로 연결될 수 있어 영역 수와 실제 부품 수가 달라집니다. 완성 예는 회색조 입력 → 이진 마스크 → 외곽 윤곽선 → 면적 필터 → 남은 영역 수 2입니다. 마스크와 박스 그림을 함께 봐야 숫자가 생긴 이유를 설명할 수 있습니다.

#### 새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 밝기 [60,90,91,200], threshold=90의 마스크를 작성하세요. 이어서 윤곽선 면적 [49,50,120], min\_area=50에서 남은 영역 수를 별도로 판단하세요.

#### 정답과 오답의 이유

마스크는 [0,0,255,255], 영역 수는 2입니다. 밝기 90을 남기는 답은 경계 비교 오류이고, 면적 50을 버리는 답은 코드의  $\geq$ 를  $>$ 로 바꿔 읽은 오류입니다. 두 흰 픽셀과 두 영역이 같은 수라는 일반 규칙은 없습니다.

#### 확인 질문과 답

1. threshold의 단위는? → 이 예의 8비트 밝기 값입니다.
2. min\_area의 역할은? → 작은 윤곽선 영역을 제외합니다.
3. 붙은 부품 둘은 반드시 두 영역인가요? → 아닙니다.

#### 짧은 완성 코드로 추적하기

제공 참고 파일 [code/05\\_threshold\\_trace.py](#)은 보존합니다. 예제 `course` 폴더에서 이 파일을 `work/05_threshold_trace.py`에 복사하고 학생 수정본은 `work`에 저장합니다. 아래는 완결된 전체 코드입니다. 파일·폴더·확장자를 확인한 뒤 같은 `course` 폴더의 터미널에서 `python -X utf8 work/05_threshold_trace.py`을 실행합니다. 저장과 실행은 별도 행동입니다.

```
import cv2

import numpy as np

gray = np.array([[80, 100, 101, 220]], dtype=np.uint8)

used, mask = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

print("threshold:", used)

print("mask:", mask.tolist())
```

예상 출력:

```
threshold: 100.0

mask: [[0, 0, 255, 255]]
```

1행은 영상 도구 cv2를 가져옵니다. 2행은 배열 도구 numpy를 np라는 짧은 이름으로 가져옵니다. 3행의 대괄호 두 겹은 한 행과 네 칸을 가진 배열이며 dtype=np.uint8은 0255의 정수로 저장한다는 뜻입니다. 4행은 함수에 입력·기준·최대값·방식을 전달하고 심표로 반환값 두 개를 나눠 받습니다. 56행의 괄호는 출력 호출, 점은 도구의 기능을 선택하는 기호입니다. tolist는 배열을 목록으로 보여 줍니다. threshold를 101로 바꾸면 마스크는 [[0,0,0,255]]입니다. cv2 설치 오류는 환경 문제로 기록하고 제공 출력을 비교합니다. 파일 확장자가 .py.txt인지도 확인합니다.

임계값은 값을 나누는 기준이고 마스크는 처리할 영역을 표시한 배열입니다. 출입 기준표처럼 픽셀마다 기준 통과 여부를 정합니다.

THRESH\_BINARY에서 밝기가 기준보다 크면 255, 작거나 같으면 0입니다. 결과의 흰색은 조건을 만족한 위치이며 물체의 의미를 이미 알아냈다는 뜻은 아닙니다.

## 입력에서 살펴볼 예

밝기 80,100,101,220 threshold=100 THRESH\_BINARY, 최대값255

80과 100은 0입니다. 101과 220은 255입니다. 같은 100을 포함하는 쪽을 주의합니다.

## 구분해야 할 반례

threshold=100이라고 밝기 100도 흰색이 되지 않습니다. 이 API의 비교는 기준보다 큰 값에 최대값을 줍니다.

스스로 설명할 질문: 밝기 100과 101이 서로 다른 결과가 되는 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

## A07 · 직접 해보기

### A07 · 임계값과 이진 마스크

**목적** THRESH\_BINARY에서 밝기가 기준보다 크면 255, 작거나 같으면 0입니다. 결과의 흰색은 조건을 만족한 위치이며 물체의 의미를 이미 알아냈다는 뜻은 아닙니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A07\\_input.md](#)

작업: [A07\\_worksheet.md](#)

## 단계

1. 네 밝기 값을 기준과 비교합니다.
2. 예상 마스크 네 값을 씁니다.
3. 기준을 60으로 바꾸면 어떤 영역이 추가되는지 예상합니다.

**예상 결과** 임계값 손계산에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** 밝기 100과 101이 서로 다른 결과가 되는 이유는 무엇인가요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

## A08 · 윤곽선과 면적 조건

윤곽선은 연결된 영역의 경계를 따라가는 점들의 목록입니다. 경계 안의 면적은 픽셀 좌표계에서 계산하며 실제 물체의 이름을 알려 주지는 않습니다.

밝은 먼지처럼 작은 영역을 제외하려면 최소 면적 조건을 함께 씁니다. 경계가 합쳐진 두 물체는 하나의 영역으로 나올 수 있으므로 개수의 해석을 제한해야 합니다.

### 입력에서 살펴볼 예

마스크: 큰 밝은 영역 2개 작은 밝은 영역 1개 min\_area=100

작은 영역의 윤곽선 면적은 16으로 제외합니다. 남은 영역을 세면 기본 예에서는 2입니다.

### 구분해야 할 반례

윤곽선 한 개가 언제나 실제 물체 한 개는 아닙니다. 물체가 붙으면 경계가 합쳐지고 그림자나 반사도 영역을 만들 수 있습니다.

스스로 설명할 질문: 검출 영역 수를 부품 개수라고 바로 확정하면 안 되는 경우는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

## A08 · 직접 해보기

## A08 · 윤곽선과 면적 조건

**목적** 밝은 먼지처럼 작은 영역을 제외하려면 최소 면적 조건을 함께 씁니다. 경계가 합쳐진 두 물체는 하나의 영역으로 나올 수 있으므로 개수의 해석을 제한해야 합니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A08\\_input.md](#)

작업: [A08\\_worksheet.md](#)

## 단계

1. 큰 영역과 작은 잡음을 구분합니다.
2. 면적 기준 100의 역할을 적습니다.
3. 붙은 부품이나 반사의 반례를 설명합니다.

**예상 결과** 면적 조건에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** 검출 영역 수를 부품 개수라고 바로 확정하면 안 되는 경우는 무엇인가요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

함께 보는 예와 더 해보기: 설정 파일로 전체 처리 읽기

[이 교시의 다른 예와 코드 해설 열기](#)

# 4교시 · 조건 비교와 검출 한계

14:00~14:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

## A10 · 한 조건만 바꾸는 비교

### 4교시 필수 개념 보충 · 한 조건씩 바꾸고 새 장면으로 시험한다

#### 실무 상황과 원리

밝은 작업대에서는 검출이 잘 되는데 조명을 줄이자 부품 하나가 사라졌습니다. 담당자는 숫자가 맞을 때까지 여러 설정을 동시에 바꾸고 싶습니다. 그러나 원인을 설명하려면 비교 조건을 관리해야 합니다. 같은 사진과 같은 크기를 유지하고 threshold만 바꾸는 실험과 min\_area만 바꾸는 실험을 나눕니다.

완성 비교표의 기본 행은 파일 P, threshold 100, min\_area 100, 관찰 영역 3입니다. 두 번째 행은 같은 P, threshold 150, min\_area 100, 관찰 영역 2라는 가상 관찰입니다. 여기서는 밝기 기준 변화의 영향을 비교할 수 있습니다. 세 번째 행에서 파일까지 어두운 D로 바꾸면 두 변화가 섞여 두 번째 행과 같은 근거로 읽지 못합니다. 값이 달라졌다는 관찰과 왜 달라졌다는 설명을 분리합니다.

기본 파일은 규칙을 만들 때 쓰고, 보류 파일은 아직 설정을 맞추지 않은 새 입력으로 씁니다. 새 입력의 기대 결과를 사람이 먼저 기록한 다음 같은 설정을 적용합니다. 기대 영역 4, 관찰 3이면 그 사례는 실패입니다. 바로 기준을 바꾸었다면 새 설정

버전으로 기존 장면도 다시 시험해야 합니다. 빈 사진, 어두운 사진, 불은 물체 사진은 서로 다른 실패를 드러냅니다. 세 합성 사진 통과를 실제 웹캠 전체 조건의 정확도라고 부르지 않습니다. 각 실행 직후 settings, mask, detected, report를 같은 실험 폴더에 복사하면 뒤 실행이 이전 실험 결과를 덮어쓰는 일을 막을 수 있습니다.

## 새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 원본 조건 (100,100)에서 한 팀은 (80,50)으로, 다른 팀은 (80,100)으로 바꾸었습니다. 밝기 기준의 영향만 설명할 수 있는 팀과 이유를 쓰고 새 시험 입력 한 종류를 제안하세요.

## 정답과 오답의 이유

두 번째 팀은 면적 기준을 유지했으므로 밝기 기준 비교에 적합합니다. 첫 팀의 결과에는 두 설정 변화가 섞였습니다. 다른 배치나 다른 조명 사진을 제안하되 무엇을 확인하고 무엇을 아직 확인하지 못하는지 적으면 인정합니다.

## 확인 질문과 답

1. 기대값은 언제 쓰나요? → 실행 전에 씁니다.
2. 새 설정이면 무엇을 재시험하나요? → 기존 입력과 새 입력을 함께 봅니다.
3. 파일 통과가 실물 통과인가요? → 아닙니다.

비교 실험은 바꾼 조건과 고정한 조건을 구분해 결과 차이의 원인을 살피는 활동입니다. 사진 두 장을 비교할 때 조명까지 함께 바뀌면 이유를 단정하기 어렵습니다.

threshold와 min\_area를 동시에 바꾸면 어느 조건 때문에 개수가 달라졌는지 알기 어렵습니다. 기본값을 적고 한 값씩 수정한 뒤 원래 값으로 돌립니다.

## 입력에서 살펴볼 예

실험1 threshold100, area100 실험2 threshold60, area100 실험3 threshold100, area2000

개수 예상은 2,3,1입니다. 실험3은 두 번째 큰 영역만 남깁니다.

## 구분해야 할 반례

실험2 결과가 좋아 보인다고 모든 카메라 조건에서 60이 최적이라는 뜻은 아닙니다. 비교에 쓰지 않은 조명과 배경에서도 확인해야 합니다.

스스로 설명할 질문: 어느 값을 고정해야 변경의 영향을 설명할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

## A10 · 직접 해보기

### A10 · 한 조건만 바꾸는 비교

**목적** threshold와 min\_area를 동시에 바꾸면 어느 조건 때문에 개수가 달라졌는지 알기 어렵습니다. 기본값을 적고 한 값씩 수정한 뒤 원래 값으로 돌립니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A10\\_input.md](#)

작업: [A10\\_worksheet.md](#)

## 단계

1. 세 설정의 결과를 먼저 예상합니다.
2. 한 값씩 바꾸고 출력과 그림을 기록합니다.
3. 원래 설정으로 복구하고 재현되는지 확인합니다.

**예상 결과** 조건 비교표에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** 어느 값을 고정해야 변경의 영향을 설명할 수 있나요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

## 각 실험의 파일을 바로 보관

실험 1 직후 `work/experiment-1` 폴더를 만들고 사용한 `settings.json`과 이번 `mask.png`, `detected.png`, `report.json`을 복사합니다. `report.json`의 `threshold.min_area`를 사용 설정과 대조한 뒤 조건을 바꿉니다. 실험 2·3도 `experiment-2`, `experiment-3`에 각각 보관합니다. 다음 실험은 같은 출력 파일을 덮어쓰므로 마지막 출력만 세 실험 결과로 제출하지 않습니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

함께 보는 예와 더 해보기: 실패한 검출에서 배우기 / 새 입력으로 같은 규칙 시험하기

[이 교시의 다른 예와 코드 해설 열기](#)

# 5교시 · 프레임 반복과 웹캠

15:00~15:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

## A14 · 웹캠 열기와 종료

카메라 캡처는 장치에서 프레임을 받아오는 연결입니다. 장비를 빌려 쓰듯 열기와 사용 뒤 반환이 한 묶음이어야 합니다.

다른 앱이 카메라를 쓰거나 권한이 없으면 열기가 실패할 수 있습니다. 열기 성공과 매 프레임 읽기 성공을 따로 검사하고 모든 종료 경로에서 자원을 해제합니다.

## 입력에서 살펴볼 예

04\_webcam.py 열기 확인 isOpened 프레임 확인 read 종료 q와 finally

기본 제작은 카메라를 열지 않았습니다. 코드 읽기와 모의 실패 경로를 비교합니다.

## 구분해야 할 반례

장치 번호0이 항상 원하는 카메라라는 보장은 없습니다. 번호를 무작정 바꿔 촬영하기 전에 장치·권한·촬영 범위를 강사가 확인합니다.

스스로 설명할 질문: 열기는 성공했는데 read가 실패할 수 있을까요?

활동 연결: [입력](#) · [기록 양식](#)

## 04\_webcam.py 전체

```
import cv2

capture = cv2.VideoCapture(0)

try:
    if not capture.isOpened():
        raise RuntimeError("카메라 열기 실패: 권한, 점유, 장치 번호를 확인하세요")

    while True:
        ok, frame = capture.read()

        if not ok:
            print("프레임 읽기 실패: 종료합니다")
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        cv2.imshow("camera-gray", gray)

        if cv2.waitKey(1) & 255 == ord("q"):
            break

finally:
    capture.release()

    cv2.destroyAllWindows()
```

실행 조건: 선택 실습: 강사가 확인한 웹캠·권한·GUI 지원 OpenCV와 사용자 동의가 있는 촬영 범위. 이번 제작에서는 실제 카메라를 열지 않았습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/04_webcam.py`

결과:

예상: 회색조 창이 갱신되고 창이 활성화된 상태에서 q를 누르면 종료.

실제 장치·창 실행은 미수행. 모의 캡처로 열기 실패·읽기 실패·q 종료와 release 호출을 검증.

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 0을 임의 번호로 계속 바꾸기 전에 운영체제의 카메라 목록과 권한을 확인합니다. 기본 실습은 03\_replay.py로 대신합니다.

오류 복구: 장치 점유 프로그램을 종료하고 권한·번호를 확인합니다. GUI가 없는 환경에서는 imshow 대신 파일 경로로 진행합니다.

## A14 · 직접 해보기

### A14 · 웹캠 열기와 종료

**목적** 다른 앱이 카메라를 쓰거나 권한이 없으면 열기가 실패할 수 있습니다. 열기 성공과 매 프레임 읽기 성공을 따로 검사하고 모든 종료 경로에서 자원을 해제합니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A14\\_input.md](#) · [04\\_webcam.py](#)

작업: [A14\\_worksheet.md](#)

#### 단계

1. 코드에서 열기·읽기·처리·종료를 표시합니다.
2. 열기 실패와 읽기 실패의 다른 경로를 따릅니다.
3. finally에서 release가 필요한 이유를 설명합니다.

**예상 결과** 장치 생명주기에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** 열기는 성공했는데 read가 실패할 수 있을까요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

함께 보는 예와 더 해보기: 프레임을 순서대로 처리 / 시연에서 본 것과 장비에서 확인할 것

[이 교시의 다른 예와 코드 해설 열기](#)

# 6교시 · 새 요구와 결과 확인

16:00~16:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

## A17 · 새 설정으로 결과 패키지 만들기

결과 패키지는 원본 이름, 설정, 출력, 확인한 사실을 함께 남긴 묶음입니다. 그림 하나만 보내면 어떤 조건으로 만들었는지 재현하기 어렵습니다.

다음 사람이 같은 결과를 확인하려면 입력과 설정이 필요합니다. 오류 입력에서는 그럴듯한 빈 결과를 성공으로 저장하지 않고 실패 이유를 분리해 남깁니다.

### 입력에서 살펴볼 예

종합 입력: holdout.png threshold=100, min\_area=100 추가 시험: missing.png

정상 입력의 기대 count는 3입니다. 없는 파일은 읽기 오류로 종료해야 합니다.

### 구분해야 할 반례

입력을 못 읽은 상태와 정상적으로 물체가 없는 상태는 다릅니다. 둘 다 count0으로 기록하면 센서 장애를 빈 공간으로 오해할 수 있습니다.

스스로 설명할 질문: 입력 실패와 정상 빈장면을 어떤 항목으로 구별할까요?

활동 연결: [입력](#) · [기록 양식](#)

## A17 · 직접 해보기

## A17 · 새 설정으로 결과 패키지 만들기

**목적** 다음 사람이 같은 결과를 확인하려면 입력과 설정이 필요합니다. 오류 입력에서는 그럴듯한 빈 결과를 성공으로 저장하지 않고 실패 이유를 분리해 남깁니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A17\\_input.md](#) · [holdout.png](#) · [empty.png](#)

작업: [A17\\_worksheet.md](#)

### 단계

1. 설정과 기대 결과를 먼저 기록합니다.
2. 새 입력을 실행 또는 제공 결과와 비교합니다.
3. 없는 파일 오류와 빈장면0을 따로 정리합니다.

**예상 결과** 종합 결과표에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** 입력 실패와 정상 빈장면을 어떤 항목으로 구별할까요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

## A18 · 결과를 근거로 설명하기

인수인계는 다음 사람이 작업을 이어갈 수 있도록 입력과 확인 결과, 남은 한계를 전달하는 일입니다. 기능 소개와 실제 검증 결과를 구분해 말합니다.

PPT 장면이나 출력 숫자만 보여 주는 발표보다 어느 조건에서 성공·실패했는지를 설명해야 수정 방향을 정할 수 있습니다. 기록의 날짜와 사용 설정도 함께 남깁니다.

### 입력에서 살펴볼 예

최종 설명3문장 1 입력과 규칙 2 직접 확인한 결과 3 미검증 장비 조건

파일 입력은 실제 실행 근거를 제시합니다. 카메라 촬영·화면 창 표시·현장 성능은 아직 시험하지 않았다고 적습니다.

### 구분해야 할 반례

완성 코드가 있다는 이유로 실시간 품질검사 시스템이 완성됐다고 말하지 않습니다. 교육용 합성 입력과 현장 운영은 검증 범위가 다릅니다.

스스로 설명할 질문: 확인한 사실과 다음 검증을 각각 한 문장으로 말할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

## A18 · 직접 해보기

## A18 · 결과를 근거로 설명하기

**목적** PPT 장면이나 출력 숫자만 보여 주는 발표보다 어느 조건에서 성공·실패했는지를 설명해야 수정 방향을 정할 수 있습니다. 기록의 날짜와 사용 설정도 함께 남깁니다.

**준비** 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A18\\_input.md](#)

작업: [A18\\_worksheet.md](#)

### 단계

1. 원본·설정·출력 파일을 제출 목록에 적습니다.
2. 세 문장으로 검증 결과와 한계를 설명합니다.

3. 새 질문 네 개에 근거를 답합니다.

**예상 결과** 제출 목록에 예상과 확인한 값과 화면을 남깁니다.

**확인 방법** 확인한 사실과 다음 검증을 각각 한 문장으로 말할 수 있나요?

**오류 해결** 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수정 전 예상: \_\_\_\_\_

확인한 결과와 이유: \_\_\_\_\_

함께 보는 예와 더 해보기: 검출 요구를 문장으로 정하기

**이 교시의 다른 예와 코드 해설 열기**

# 오늘 배운 내용 확인

| 확인할 것    | 배점  | 충족 기준                                   |
|----------|-----|-----------------------------------------|
| 배운 내용 설명 | 20점 | 사용한 개념과 선택한 이유를 예로 설명한다.                |
| 결과물 완성   | 40점 | 오늘 필수 활동의 파일·표·기획 내용을 빠짐없이 완성한다.        |
| 결과 확인    | 30점 | 예상과 결과를 비교하고 다른 입력 한 가지로 다시 확인한다.       |
| 저장과 전달   | 10점 | 파일 이름과 실행·읽는 방법을 함께 적어 다른 사람이 확인할 수 있다. |

각 항목: 모두 충족하면 배점 전부, 일부 충족하면 배점의 절반, 근거가 없으면 0점. 오류를 발견하고 고친 기록도 확인 근거로 사용합니다.

남길 결과: 종합 결과표 / 실패 기록 / 제출 목록 / 입력·설정·결과 설명

## 세 가지 확인 질문

1. 이미지 읽기 실패와 정상적으로 읽었지만 검출 수가 0인 경우는 어떻게 다른가요?

내 설명: \_\_\_\_\_

2. 오늘 필수 활동에서 값을 한 곳 바꾼 예를 고르세요. 바꾸기 전 예상과 확인한 결과를 설명해 보세요.

내 설명: \_\_\_\_\_

3. 마지막 결과물의 파일 이름과 읽거나 실행하는 방법을 적어 보세요. 다른 사람이 확인할 수 있도록 한 문장씩 안내합니다.

내 설명: \_\_\_\_\_

## 더 해보기 · 추가 확인 문제

### 설명으로 확인하는 질문

1. `shape`가 `(480, 640, 3)`이라면 너비와 채널 수는 각각 무엇인가요? 숫자의 순서를 함께 설명하세요.

2. `THRESH_BINARY`의 기준이 100일 때 밝기가 정확히 100인 픽셀은 어느 쪽으로 처리되나요? 코드의 비교 규칙을 근거로 설명하세요.

3. 붙어 있는 표식 두 개가 윤곽선 하나로 검출됐다면, 검출 수만 보고 물체가 하나라고 결론 내릴 수 있나요? 다음에 확인할 것을 적으세요.

4. 모의 캡처에서 `release` 호출을 확인하면 실제 카메라 드라이버 동작까지 검증한 것인가요? 시험 범위를 설명하세요.

## 새 상황으로 옮겨 보는 다른 상황에 적용하는 문제 · 복습·도전

교시당 한 문제이며 기존 필수 평가 후 필요에 따라 선택합니다. 독립 작성 또는 판단을 먼저 기록한 뒤 해설을 확인합니다. 설치 실패나 장비 부재는 개념 오답과 구분합니다.